EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Machine Learning Lab

Course code: CSL604

Year: TE3 SEM: VI

Experiment No. 05
AIM:- To implement Hebbian Learning algorithm.
Name: Satyam Maurya
Roll Number: 62
Date of Performance:16-02-26
Date of Submission:23-02-26

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty : Ms. Nivedha Raut

Signature :

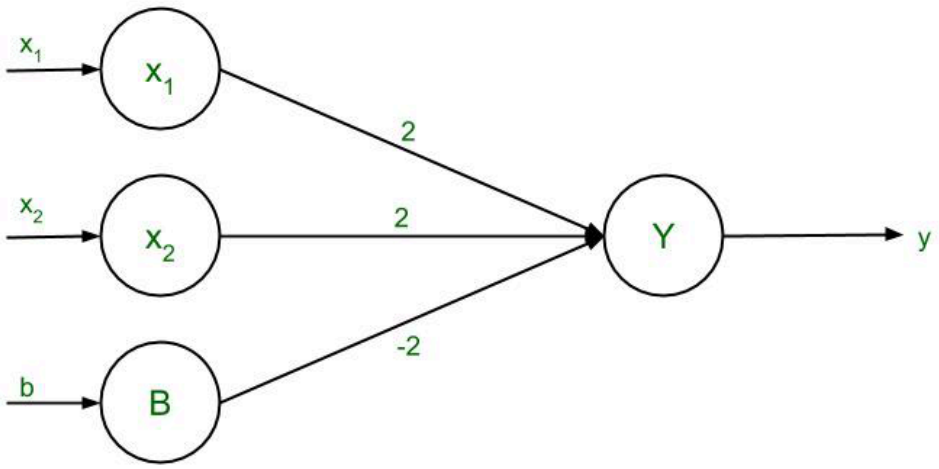
Date :

Aim - To implement Hebbian Learning algorithm.

Objective - To simulate the biological process of learning by implementing a Hebbian neural network.

Theory –
Hebbian learning is a fundamental concept in neural networks and the field of Machine Learning, inspired by how biological neurons are believed to adapt during learning. Proposed by Donald Hebb in 1949, the theory is often summarized by the phrase "cells that fire together, wire together." It states that the connection (synaptic weight) between two neurons strengthens when they are activated simultaneously. In computational terms, if an input neuron and an output neuron are both active at the same time, the weight between them increases proportionally. This learning rule is unsupervised, meaning it does not require labeled data or external correction, and instead relies solely on local information—specifically, the activity of connected neurons. Mathematically, the change in weight is typically expressed as the product of the input and output signals, scaled by a learning rate. Hebbian learning plays an important role in pattern recognition, associative memory, and feature extraction, and it forms the basis for more advanced models such as competitive learning and self-organizing maps. Despite its simplicity, it has limitations, such as unbounded growth of weights, which often requires normalization techniques to maintain stability.

w_i (new) = w_i (old) + x_i y
b (new) = b (old) + y



Code:

```
import numpy as np

# 1. Define the Input Data (X) and Target Labels (y)
# Using Bipolar inputs: 1 for True, -1 for False
# Format: [x1, x2, bias]
X = np.array([
    [1, 1, 1],   # True OR True = True
    [1, -1, 1],  # True OR False = True
    [-1, 1, 1],  # False OR True = True
    [-1, -1, 1]  # False OR False = False
])

y = np.array([1, 1, 1, -1])

# 2. Initialize weights to zero
weights = np.zeros(3)
learning_rate = 1

print("Initial Weights:", weights)
print("-" * 30)

# 3. Hebbian Learning Rule Training
def train_hebb(X, y, weights):
    for i in range(len(X)):
        # Hebbian rule: Change in weight = input * output
        # Δw = x * y
        delta_w = X[i] * y[i]
        weights = weights + delta_w

        print(f"Step {i+1}: Input {X[i][:2]} | Target {y[i]}")
        print(f"New Weights: {weights}")
    return weights

final_weights = train_hebb(X, y, weights)

# 4. Testing the model
print("-" * 30)
print("Final Trained Weights:", final_weights)

def predict(inputs, weights):
```

```
    # Summation (dot product)
    net = np.dot(inputs, weights)
    # Activation function (Signum function)
    return 1 if net >= 0 else -1

print("\nTesting the OR Gate:")
for i in range(len(X)):
    prediction = predict(X[i], final_weights)
    print(f"Input: {X[i][:2]} | Predicted: {prediction} | Actual: {y[i]}")
```

Output:

```
Initial Weights: [0. 0. 0.]
------------------------------
Step 1: Input [1 1] | Target 1
New Weights: [1. 1. 1.]
Step 2: Input [ 1 -1] | Target 1
New Weights: [2. 0. 2.]
Step 3: Input [-1  1] | Target 1
New Weights: [1. 1. 3.]
Step 4: Input [-1 -1] | Target -1
New Weights: [2. 2. 2.]
------------------------------
Final Trained Weights: [2. 2. 2.]

Testing the OR Gate:
Input: [1 1] | Predicted: 1 | Actual: 1
Input: [ 1 -1] | Predicted: 1 | Actual: 1
Input: [-1  1] | Predicted: 1 | Actual: 1
Input: [-1 -1] | Predicted: -1 | Actual: -1
```

Conclusion:

The **Hebbian learning rule** represents a fundamental biological principle in neural networks, grounded in the concept that the synaptic strength between two neurons increases when they consistently fire in synchronization. By mathematically translating this "cells that fire together, wire together" logic into a weight-update mechanism, the rule allows a network to

learn associations and patterns without the need for complex error-correction signals like backpropagation. While its simplicity makes it an elegant model for unsupervised learning and pattern recognition, its tendency for weights to grow indefinitely limits its use in modern deep learning unless paired with normalization techniques. Ultimately, Hebbian learning remains a cornerstone of computational neuroscience and serves as a vital bridge between biological brain functions and the development of self-organizing artificial systems.